Blender to Universal Robots (UR) Real-time Control Interface

1. Introduction

This project provides a Python script that enables users to control a Universal Robots (UR) arm in real-time through Blender's 3D interface. Using Socket communication (TCP/IP), the script translates object coordinates in Blender into movement commands for the robot arm.

Key Features:

- **Real-time Synchronization:** Synchronizes the movement of a Blender Empty to the robot TCP.
- **Safety Bounding Box:** Automatically returns the robot to a home position if the target moves outside a defined area, preventing collisions.
- **Smooth Interpolation:** Built-in smoothing algorithms ensure fluid robot movements and prevent jitter.
- **UI Panel:** Provides an intuitive control panel within Blender's sidebar.

---

2. Hardware & Network Setup

To ensure low latency and stability, this script is designed to work with a direct wired ethernet connection.

Prerequisites:

- A Universal Robot UR3e and control box.
- A Computer (Windows or Mac) with Blender installed.
- Ethernet Cable

Step 1: Connect Hardware

Connect one end of the Ethernet cable to the Ethernet port on your computer and the other end to the Ethernet port inside the UR Control Box.

Step 2: Check Robot IP Address

Check its current IP address and configure the computer to match it.

- On the UR Teach Pendant, go to Settings > System > Network.
- Note down the current IP Address displayed (e.g., 192.168.0.100).
- Note down the Subnet Mask (usually 255.255.255.0).

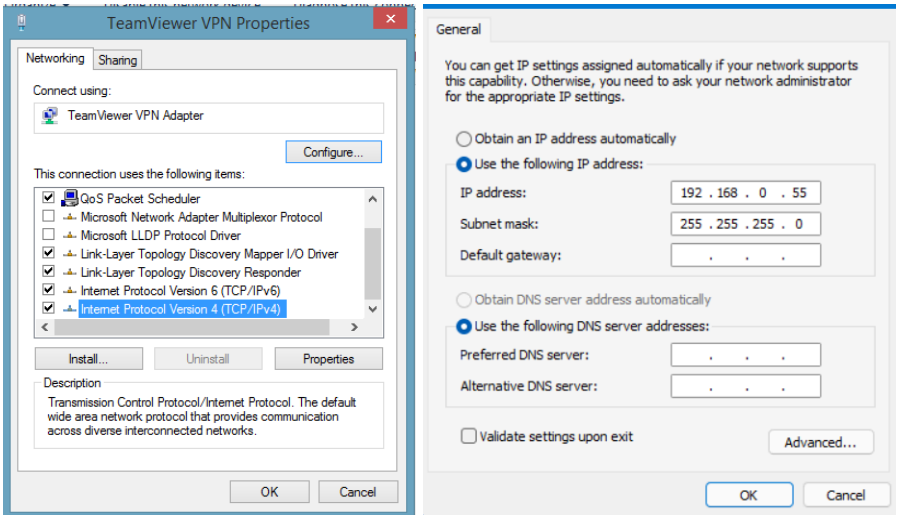
Step 3: Configure Computer IP

Your computer must be on the same network segment as the robot. This means the first three parts of the IP address must match, but the last part must be different.
Example:

- Robot IP: 192.168.0.100 (This IP can be verified on the UR Robot Teach Pendant)
- Computer IP: 192.168.0.101 (Set this on your PC/Mac)

For Windows Users:

- Go to Control Panel > Network and Sharing Center > Change adapter settings.
- Right-click the Ethernet adapter connected to the robot and select Properties.
- Select Internet Protocol Version 4 (TCP/IPv4) and click Properties.

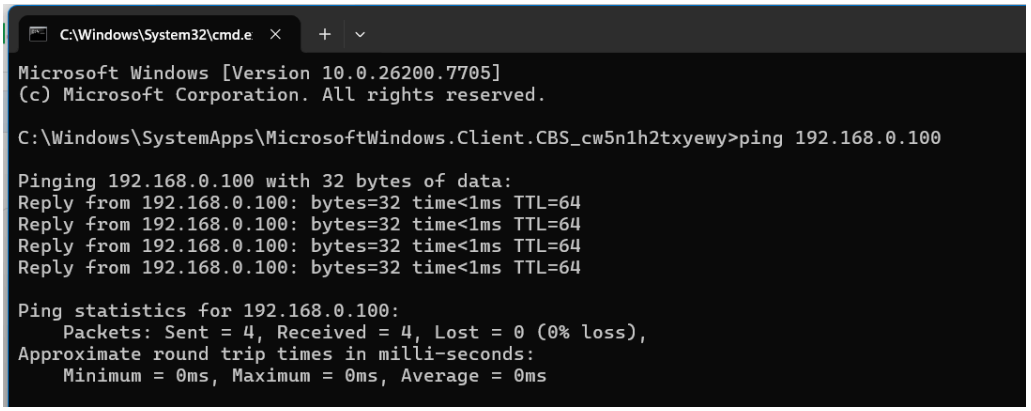


- Select Use the following IP address:
    - IP address: 192.168.0.101 (Or any number different from the robot's)
    - Subnet mask: 255.255.255.0
- Click OK.

For Mac Users:

- Go to System Settings > Network.
- Click on the Ethernet interface connected to the robot.
- Click Details... > TCP/IP.
- From the Configure IPv4 dropdown, select Manually.
- Enter the settings:
    - IP Address: 192.168.0.101
    - Subnet Mask: 255.255.255.0
- Click OK.

Step 4: Test Connection



Open your **Terminal** (Mac) or **Command Prompt** (Windows) and type:

ping 192.168.0.100

(Replace the IP with your robot's actual IP).

If you receive a "Reply" or successful packets, the connection is established.

Step 5: Enable Remote Control

Before running the live sync script in Blender, you must switch the UR3e robot from Local Control to Remote Control on the Teach Pendant. If the robot remains in Local mode, it will reject all movement commands sent from Blender.

How to switch:

1. On the UR Teach Pendant screen, look at the top right corner.
2. Tap the Control Mode icon (usually showing a small Teach Pendant icon).
3. Toggle the setting from **Local Control** to **Remote Control**.

4. The icon should change to a small PC/Network symbol, indicating the robot is now ready to receive external network commands.

---

3. Blender Scene Setup

The script relies on specific object names to function. Ensure your Blender scene contains the following objects (or update the CONFIGURATION section in the script):

Target Object (pen_Bone):

The input object (Bone or Empty) that you will animate or move to control the robot. you need to change the name in the script if you have any other motion capture data.

Controller Object (mocap_cleaned):

The object driven by the script logic. This usually follows the Target but respects the Bounding Box limits.

Output Object (TCP_Empty):

Represents the robot's Tool Center Point (TCP). The script reads this object's world coordinates to send to the robot.

Bounding Box (bounding_box):

A Mesh object that defines the safe working area. If the target leaves this box, the robot returns home.

---

4. Script Functions Overview

Core Logic

- Tracking: The script reads the position of the “brush_Bone.002” (motion capture data).
- Safety Check: It checks if the target is inside the bounding_box.
    - Inside: The robot follows the target.
    - Outside: The robot smoothly returns to the Start_Position.
- Communication: A dedicated thread sends the coordinates of TCP_Empty to the robot via Port 30003.

UI Panel Parameters

Located in the Blender Sidebar (N-Panel) under the UR Control tab:

- Robot IP: Enter the Robot's IP address here (default is 192.168.0.100).
- Follow Logic: Enable/Disable the Bounding Box safety logic.
- Max Speed: The value corresponds to Meters per Second (m/s) in the Blender coordinate system (lower values = slower move).
- Rot Smooth: Controls rotational smoothing (lower values = smoother/slower rotation).
- Reset Defaults: If you have experimented with the IP address, speed, or smoothing settings and want to return to the original safe configuration, simply click the Reset Defaults button. This will instantly revert the IP to 192.168.0.100 and the Speed to the safe 0.15 limit.

---

5. How to Use

- Open your Blender project and go to the Scripting workspace.
- Load the Python script provided in this repository.
- Check your computer is connected by robot already (open your Terminal (Mac) or Command Prompt (Windows) and type:

  ping 192.168.0.100

- Click the Run Script button (Play icon).
- In the 3D Viewport, press N to open the sidebar and find the UR Control tab.
- Click **START LIVE SYNC** (or press F2).
  - The robot will move to the initial start position.
- When the animation has finished, click the button again or press F2 to **STOP & RETURN HOME.**

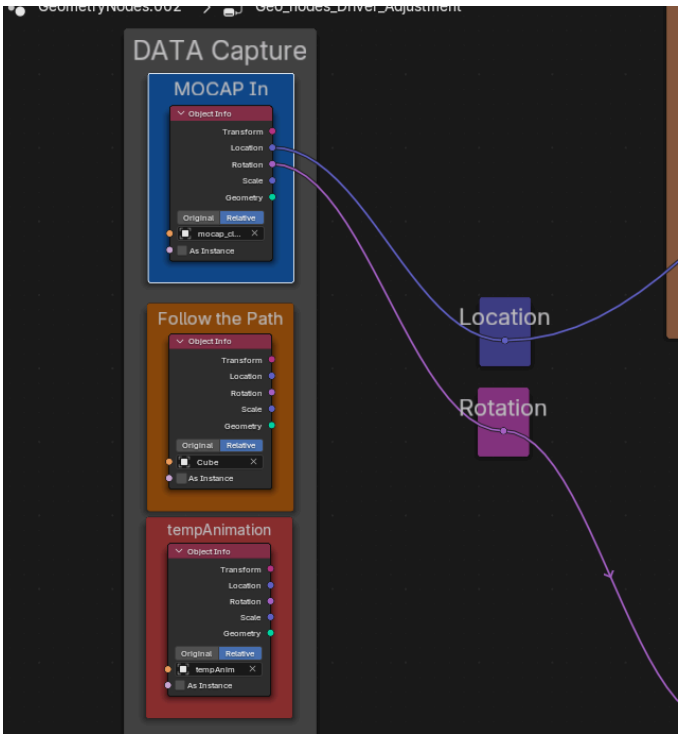
6. Advanced Configuration: Geometry Nodes Driver

While the Python script handles the connection, the *"Geometry_Nodes_Driver"* object acts as the "Brain" of the operation. It processes the raw animation data (from Motion Capture or a Path) before sending it to the robot.

This setup allows you to fine-tune offsets, limit movement ranges, and dampen rotation without changing your original animation data.

How to Access the Controls

- In the Blender 3D Viewport, select the object named *"Geometry_Nodes_Driver"*.
- Go to the Geometry Nodes tab (top menu bar) or open a Geometry Node Editor window.
- You will see a node graph. Look for the Green Value Nodes. These are your control dials.



a. Switching Input Sources (Mocap, Path, or Keyframe)

The system is designed to accept different types of inputs. In the "DATA Capture" frame (left side of the graph), you will now see three main input blocks:

MOCAP In: Uses data from the mocap_cleaned object (live or recorded motion capture).
Follow the Path: Uses data from an object (e.g., a Cube) following a curve.
tempAnimation: Uses data from the tempAnim object for your custom manual keyframe animations.
To switch the source:

- Locate the **Location** and **Rotation** output sockets (the blue and purple dots) on your desired input node (for example, tempAnimation).

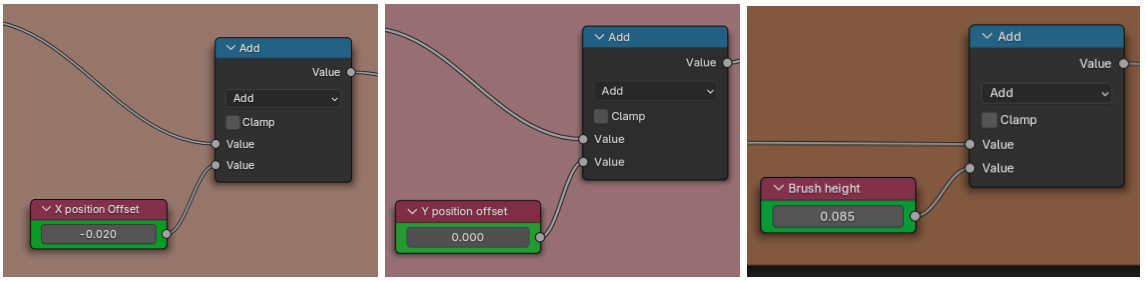
- Click and drag a wire from these sockets to the corresponding **Location** and **Rotation** reroute nodes (the floating blue and purple boxes to the right), replacing the existing connected wires.

The robot will now ignore the previous data and follow your newly connected animation source instead.

b. Adjusting Position Offsets

If your robot is moving correctly but is slightly misaligned (e.g., too far left or too high), you don't need to move the whole scene. You can use these offset nodes to shift the final output.

Look for the Green Nodes labeled:



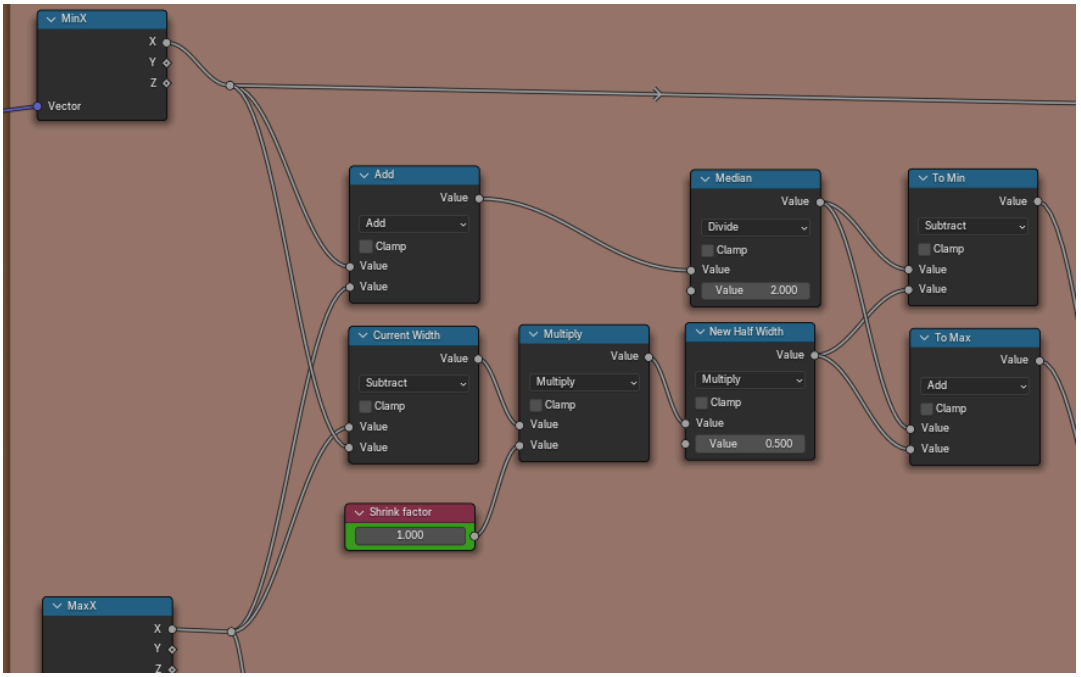
X position Offset: Shifts the robot along the X-axis.
Y position offset: Shifts the robot along the Y-axis.
Brush height (Z Offset): Adjusts the vertical height.

*Important Note: If you change this Z offset value, you must also update the **TOOL_OFFSET** variable in the Python script to match. The formula is simply:*

*If your Brush height (Z Offset) = **0.085**,*
*And your ROBOT_HOME_POS (Z Value) = **0.304**,*
*Then, TOOL_OFFSET = 0.304 - 0.085 = **0.219**.*

c. Shrink Factor

To prevent the robot from reaching too far, the setup includes a "Range Warp" logic.
Look for the Green Node labeled Shrink factor.



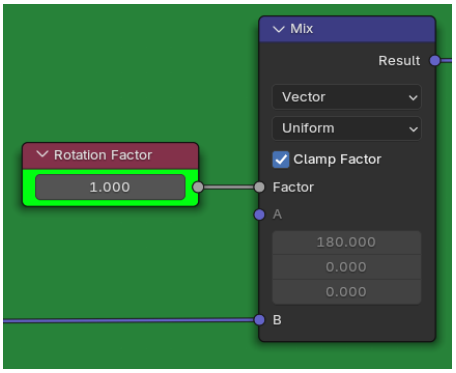
If value 1.000: Uses the full range of the bounding box.

If value < 1.000 (e.g., 0.5): Shrinks the movement area towards the center, the drawing small and centered, regardless of how big the original motion capture movement was.

d. Adjusting Rotation

Look for the Green Node labeled Rotation Factor.

This acts as a multiplier for the rotation angles (Rx, Ry, Rz).



If value is 1.000: 100% Rotation. The robot matches the input rotation exactly.

If value is 0.500: 50% Rotation. The robot rotates only half as much as the input. This is smoother and safer.

If Value IS 0.000: No Rotation. The robot keeps the tool facing down (or its default orientation) and only moves in position (X, Y, Z).

7. Visualizing the Drawing (Digital Canvas)

Before sending commands to the real robot, you can preview the drawing result in Blender using the Dynamic Paint system. To make this preview accurate, the digital canvas must match the size and position of your real-world paper.

How to Adjust the Canvas

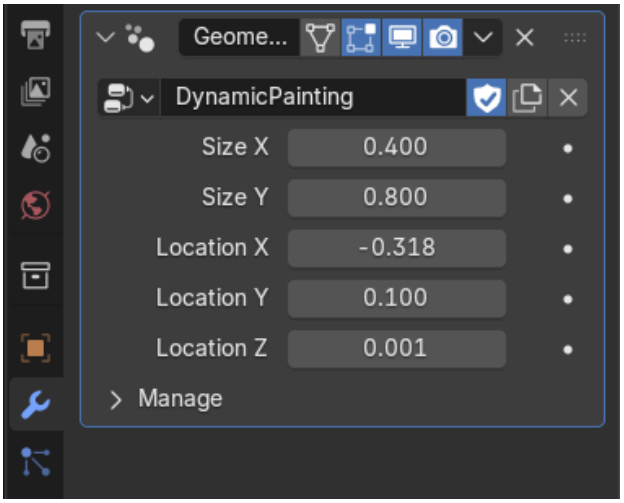
- **Locate the Object:** In the Outliner (top-right usually), find the Collection named Canvas_Dynamic_Painting and select the object named Canvas.
- **Open Modifiers:** Go to the Properties Panel (bottom-right) and click on the Modifiers Tab.

Adjust Parameters:

Inside the Modifier settings for the Canvas, you will find exposed values to control its properties. You can modify:

Size (Length (X) / Width (Y)): Change these values to match the physical size of the paper you are using.

Location (X, Y, Z): Adjust these to move the digital canvas so it sits exactly where the robot's brush (the TCP_Empty) is touching.



8. Script Configuration (Advanced)

While most settings can be controlled via the Blender UI panel, some core configurations are hard-coded in the Python script for safety and stability. You can find these variables at the top of the script text.

```
# Kinematics & Safety
DEF_MAX_SPEED = 0.15
DEF_ROT_SMOOTH = 0.10
DEF_LOGIC_ENABLE = 'YES'

# Physical Robot Home Position (Safe parking spot: X, Y, Z in meters)
ROBOT_HOME_POS  = [-0.2983, 0.1314, 0.304]

# Visual Tool Offset (Z-axis height in Blender)
# IMPORTANT: If you change to a longer or shorter pen, you MUST update this value
# to match the new tool tip's Z-height in your Blender scene.
TOOL_OFFSET = 0.219

# ServoJ Constants
INTERNAL_T = 0.03
INTERNAL_LH = 0.03
INTERNAL_GAIN = 300
```

a. Customizing the Safe Home Position & Tool Offset

To protect your physical hardware and account for the exact length of your attached tool (like a pen or marker), the script uses a smart dual-variable system. The physical robot will automatically return to its safe parking spot when:

1. The script initializes (Moving to First Frame).
2. The tracking target moves outside the safe Bounding Box.
3. Click the "STOP & RETURN HOME" button.

Default Value:

ROBOT_HOME_POS = [-0.2983, 0.1314, 0.304]

If you change your physical tool (for example, swapping a short pen for a longer marker), you must update these values to prevent collisions and keep your 3D animation accurate:

Update **ROBOT_HOME_POS** (Z-axis): Increase or decrease the third number (0.304) to ensure the real robot parks at a safe physical height with the new tool attached.
Update **TOOL_OFFSET**: Change this value (0.219) to match the new Z-height of your tool tip inside your Blender scene. This ensures the virtual controller stays perfectly aligned with your digital canvas.

b. Internal Control Constants Explanation

These parameters define the low-level behavior of the robot's movement. It is recommended to keep the default values.

INTERNAL_T (Time Step)
Defines the time interval (in seconds) between each command sent to the robot.

- Value too LOW: The system requires a perfect network connection. Any slight delay will cause the robot to stutter or disconnect.
- Value too HIGH: The movement will become choppy, losing its fluidity.

INTERNAL_LH (Look-ahead Time)
Determines how much the robot smooths out the incoming movement data.

- Value too LOW: The movement becomes raw and jittery, reacting to every small noise in the sensor data.
- Value too HIGH: The movement becomes very smooth but introduces a noticeable delay (lag) between your input and the robot's action.

INTERNAL_GAIN (Proportional Gain)
Controls the strength and stiffness of the motors when moving to a target position.

- Value too LOW: The robot becomes "loose." It may not reach the exact target position accurately or may drift.
- Value too HIGH: The robot becomes overly aggressive. This causes vibrations, loud buzzing noises, and will trigger a Protective Stop (safety shutdown).

Recommendation:
Please keep these default values unchanged.
These parameters have been tuned to balance low latency with smooth movement. Modifying them (especially INTERNAL_T or INTERNAL_GAIN) can cause the robot to vibrate, move jerkily, or trigger a Protective Stop due to communication desynchronization. Only advanced users familiar with URScript servoj commands should attempt to tune these.